

names. For example, the above program uses the symbols μ and σ as variable names to store the mean and standard deviation, respectively. In the Julia REPL, you can get Unicode math symbols by typing the symbol names in LaTeX followed by typing the *tab* key. For example, the variable name μ can be generated by typing ‘\mu’ (LaTeX for μ) followed by the *tab* key in the Julia REPL. These types of variable names make the code more expressive, easier to read, and closer to the mathematical notation used in formulas and equations. The Julia program can be executed with the command ‘*julia pgm1.jl*’. Figure 6 shows the execution and the output of the Julia program *pgm1.jl*.

By now, we are quite familiar with JupyterLab and Jupyter Notebooks. One huge benefit of Julia is that it can be used with Jupyter Notebooks, allowing users to leverage the strengths of both technologies. Julia is a package that enables the use of Julia within a Jupyter Notebook environment, making it easy to work with data, perform computations, and visualise results. In this series, we have already installed and used JupyterLab. Now, let us explore how Julia can be installed and used as a kernel in Jupyter Notebooks. Once you have installed Julia, open the Julia REPL and enter the following two commands on the Julia prompt to add the IJulia package: ‘*using Pkg*’ and ‘*Pkg.add("IJulia")*’. The execution of these two commands will install the IJulia package on your system. Now, execute the command ‘*Jupyter lab*’ to access Julia in a Jupyter Notebook. Figure 7 shows the option to choose Julia (marked inside a blue box) as the kernel for a Jupyter Notebook. As mentioned earlier in this series, JupyterLab can support various kernels. As you can see, my system supports not only Julia but also Python and SageMath kernels.

```
# julia pgm1.jl
Array: [71, 99, 22, 59, 28, 84, 70, 25, 71, 79]
Mean: 60.8
Standard Deviation: 26.848546412132713
```

Figure 6: Output of the Julia program *pgm1.jl*



Figure 7: Julia in JupyterLab

```
start = time()
s = factorial(BigInt(200000))
finish = time()
println("Julia: ", finish - start, " Seconds")
#println(s)
Julia: 0.032067060470581055 Seconds

import time
import math
start = time.time()
s = math.factorial(200000)
finish = time.time()
print("Python: ", finish - start, "Seconds")
#print(s)
Python: 0.3572051525115967 Seconds
```

Figure 8: Comparing the execution speeds of Julia and Python

Let us wrap up our discussion of Julia by comparing the execution speeds of Python and Julia.

Figure 8 shows programs to find the factorial of the number 200000, using both Julia (shown at the top of the figure) and Python (shown at the bottom of the figure). Just in case you forgot what a factorial is, the factorial of 5 (denoted as 5!) is 120 (5x4x3x2x1). The programs are straightforward because both Julia and Python have a function called *factorial()* to find the factorial of a number. I commented out the line of code to print the answer obtained because the final answer has nearly a million digits (973,351 to be exact), which would require 235 A4-size

pages to print. Figure 8 also shows the amount of time that the Julia and Python programs took to complete the calculation. It is evident that Python took ten times longer than Julia. While there are strategies to optimise the code, this is the most apparent code that comes to the mind of most programmers, and Julia is the undisputed victor here.

In conclusion, I would like to emphasise the speed of Julia — it produced a number with almost a million digits in less than 33 milliseconds on my laptop (which has modest specifications).

Let us now pause for a moment to consider the remarkable advances in computing power and scientific progress over the last century. Although it is a bit saddening that we can only imagine the technology that will exist 100 years hence, it brings

us peace and happiness to know that our children will have a brighter future due to the rapid progress in science and technology.

A quick recap

Now, let us briefly go over the topics covered in this series, along with some key topics that were either left out or given only brief coverage. In this series, I attempted to introduce maths, particularly linear algebra and probability, as needed. However, it is important to note that our discussions were not highly formal or extensive. I want to emphasise once again that understanding maths is crucial for mastering the techniques required for developing AI-based solutions,